Rapid Tech Insights

ENGINEERING REPORT
Rapid Inventory · Fixes, verification and release readiness

Root Cause Analysis — Rapid Inventory

Rapid Tech Insights · Engineering report · 29 October 2026 · Version 1.1

This report explains what was wrong with Rapid Inventory, how we proved it, what we changed, and how we know the changes work. Every number here comes from something we actually ran. Where something has not been tested, we say so.

Short on time? Read **A** (summary) and **K** (what's left). SOLUTIONS.md is the one-page version. The release report and all raw evidence are in `release/`.

A. Summary

Users reported three problems. We also refactored order placement and added low-stock alerts.

What users saw	What was actually wrong	What we did	Verified?
Customers could buy more stock than existed	Two orders could read the same stock level at the same moment, and each overwrote the other's update.	Check and deduct stock in a single database statement, inside one transaction. Added a database rule that stock can never go below zero.	Yes
The inventory page was slow	Each page load ran **202** database queries: one extra query per product, just to fetch its category name.	Fetch everything in one query: **2** queries per page.	Yes: 225 ms → 12 ms
Other users' live boards didn't update	Stock updates were sent only to the person who placed the order.	Send updates to everyone watching.	Yes, in two real browsers
Order logic was crammed into the controller	One class did validation, stock logic, saving and notifications. An unused copy of the same logic sat elsewhere and had drifted out of date.	The order service now does the work; the controller just handles HTTP.	Yes: identical API behaviour
No visible low-stock warnings	The server already sent alerts, but only the Live board page showed them.	A toast alert now appears on every page, for every user.	Yes

While refactoring we found one more bug: cancelling the same order several times at once could return its stock several times. That is fixed too.

Still open: live updates need extra infrastructure (a SignalR backplane) if more than one API server runs; alerts are not guaranteed to arrive; a client retry after a timeout can create a duplicate order; and the walkthrough video has not been recorded yet. Details in section K.

B. Environment and how to reproduce

Piece	Version we tested
API	.NET 9 (ASP.NET Core 9.0.22), EF Core 9.0.10
Database	SQL Server 2022 (16.0.4305.5), in Docker
Front end	Angular 20.3, SignalR client 10.0.11
Host	Windows 11, Docker 29.8, Chrome 156

Start it: `docker compose up --build`. The first start downloads about 3 GB and loads 2,000 products and 250,000 historical orders. After that:

- App: http://localhost:4200
- API: http://localhost:5080
- API docs: http://localhost:5080/swagger

Run the tests:

```
dotnet test Inventory.sln                                          # 42 backend tests, real SQL
Server
cd frontend && npx ng test --watch=false --browsers=ChromeHeadless  # 17 front-end tests
node seed/two-browser-check.mjs FPR-000021                         # two real browsers, live
stack
```

The backend tests create and delete their own throwaway database. They read the connection string from `ConnectionStrings__DefaultConnection`.

Things that got in the way (and how we handled them):

- **No Git history.** The project came without any. We committed the untouched code first (`fc3c873`), so every change can be reviewed as a diff.
- **Port 1433 clash.** A local SQL Server Express on the test PC already used port 1433, so tools on the PC were talking to the wrong database. The containers were not affected. We exposed the Docker database on port 14333 as well, using a local override file that is not in the repo.
- **Migrations and hot reload.** `dotnet watch` reloads code but doesn't run new migrations; we restarted the API container after adding one.
- **Stress-script counts.** The repo's stress script miscounts its own results on Windows. We used database counts as the truth, and added `seed/load-test.mjs`, which counts exactly.

C. Root causes, one by one

C1. Overselling when orders arrive together

What users saw. More units were sold than were in stock.

What was wrong. The old code did three separate steps: read the stock, check it, then write a new number.

```
SELECT ... Stock FROM Products WHERE Id = @id                    -- both requests read 1
UPDATE Products SET Stock = @newStock WHERE Id = @id             -- both write 0
INSERT INTO Orders ... ; INSERT INTO OrderLines ...              -- both orders saved
```

When two orders arrive together, both read "1 left", both pass the check, and both write "0". Stock never goes negative. Instead you get more units in orders than were ever taken off the shelf. The stock update and the order were also saved separately, so if the second item in an order failed, the first item's stock was still taken.

Evidence (original code, real SQL Server):

- Two API instances racing for the last unit: **both succeeded**.
- 40 orders against 10 in stock: **all 40 accepted**, and stock showed 5.
- The repo's own stress script: 40 units sold while stock only dropped from 30 to 13.
- A failed second item left the first item's stock deducted (5 → 3).
- If the live-update notification failed, the user got an error for an order that had actually been saved, inviting them to place it again.

What we changed (`OrderService.cs`):

- The check and the deduction are now **one statement**: `UPDATE Products SET Stock = Stock - @qty WHERE Id = @id AND Stock >= @qty` . If it changes no rows, there wasn't enough stock: the request gets a 409 and everything is undone.
- The stock change and the order are saved in **one transaction**: all or nothing.
- A database rule, `CK_Products_Stock_NonNegative` , makes negative stock impossible, whatever any code does.
- Notifications go out only **after** the order is saved. If sending one fails, we log it and still tell the customer their order succeeded.

Verified:

- Last-unit race: one success, one "insufficient stock".
- 40 orders against 10 in stock: exactly 10 accepted, 30 refused, stock 0.
- Two real API processes against 80 units: 41 + 39 = 80 accepted, stock 0.
- Stress script: 29 units in stock, exactly 29 sold.
- All 12 concurrency tests pass.

Still open: there is no "idempotency key", so if a customer's request times out and they retry, a second order can be created. In theory, two multi-item orders that lock the same products in opposite order could block each other: SQL Server would cancel one cleanly (no bad data), but the user would see an error. We haven't seen this happen.

C2. Slow inventory page

What users saw. The inventory page was slow to load and search.

What was wrong. The page asks for 200 products. The old code fetched the 200 products, then asked the database for each product's category name **one at a time**: 202 queries per page load. The 250,000 orders weren't involved at all.

Evidence. The API's own SQL log shows exactly 202 queries for one page load. An automated test counted the same.

What we changed. One query that joins products to categories and fetches only the columns the page shows. We added no index: the database plan showed it didn't need one.

Verified (same data, same conditions, 50 requests each):

	Before	After
Inventory page (200 rows), typical	225 ms	**12 ms**
Inventory page, slowest 5%	261 ms	16 ms
Search for "Milk"	95 ms	23 ms
Queries per page load	202	2
Data returned	28,978 bytes	identical

Still open: search uses "contains" matching, which has to look at every product. That's fine for 2,000 products, but it would need full-text search for a much bigger catalogue.

C3. Live board not updating for other users

What users saw. One person orders; everyone else's screen keeps showing the old stock.

What was wrong. We followed the whole path: order saved → server sends `StockUpdated` → browsers receive it → page updates. The connection, URLs and event names were all fine. The problem was who the message went to:

```
var clients = requesterConnectionId is null
    ? hub.Clients.Group("stock-watchers")      // everyone
    : hub.Clients.Client(requesterConnectionId); // only the person who ordered
```

The inventory page always sends its connection ID with an order, so only the buyer was told. Restocks don't send one, which is why restocks did reach everyone. A second, smaller issue: if the connection dropped for more than about 40 seconds, the page gave up and never reconnected.

Evidence. In two real browsers, after an order in Browser A, Browser B stayed at 41 while the database said 40.

What we changed. Updates now always go to everyone watching. The page keeps trying to reconnect, and reloads its data once it's back.

Verified:

- In two separate browser sessions, Browser B showed the new stock about 4 ms after Browser A, without a refresh.
- A failed order sent nothing at all.
- 5 automated SignalR tests pass.

Still open:

- Reconnecting after a real outage is tested only in unit tests, not live.
- With **more than one API server**, users only hear about changes made on the server they're connected to. That needs a backplane, such as Redis.

C4. Too much logic in the order controller

What was wrong. `OrdersController` did everything: validation, stock checks, saving, transactions and notifications. An `OrderService` existed but nothing used it, and it had drifted out of date (it still had the overselling bug). Cancelling also had a race: when 10 cancels of the same order arrived together, **6 of them** returned the stock.

What we changed. `OrderService` now handles placing, cancelling and restocking. The controller only turns results into HTTP responses. We didn't add any new layers or interfaces.

How we know nothing broke. Before touching the code, we wrote 13 tests that pin down the API's exact behaviour: status codes, error messages and response bodies. They passed on the old code, and they pass, unchanged, on the new code.

C5. Low-stock alerts

What was wrong. The server already sent a `LowStockAlert`, but only the Live board page displayed it. Not the inventory page, where orders are actually placed. A second alert for the same product could also confuse the board's list.

What we changed. A small alert ("toast") now appears on every page, can be dismissed, and is announced to screen readers. Each alert has its own ID, so repeats are handled cleanly.

The rule: an alert fires only when an order takes stock from **at or above** the reorder level to **below** it. Landing exactly on the level doesn't alert. Orders made while it's already low don't repeat the alert. A restock re-arms it.

Verified:

- 8 automated rule tests pass.
- In two live browsers: no alert at exactly 15; at 14, the alert appeared in **both** browsers; a failed order added nothing.
- The actual message received was `{"productId":41,"productName":"Sukuma Wiki 500g","stock":14,"reorderThreshold":15}`.

Still open:

- Alerts are not stored. Someone who opens the app later won't see earlier alerts.
- On the inventory page the toast slightly overlaps the heading counter (cosmetic).

D. How the overselling fix works

The scenario. Two orders for the last unit arrive in the same millisecond, on two different API servers.

1. Both servers send the same statement: `UPDATE Products SET Stock = Stock - 1 WHERE Id = @id AND Stock >= 1`.
2. Before SQL Server can check `Stock >= 1`, it has to lock that product's row. Only one request can hold the lock, so one wins. Which one is up to SQL Server, not our code.
3. The winner sets stock to 0, saves its order and commits.
4. The loser was waiting for the lock. When it gets it, SQL Server checks the condition again against the **saved** row: 0 is not ≥ 1, so nothing changes.
5. Our code sees "0 rows changed", replies **409 Insufficient stock**, and undoes the loser's transaction. No order, no stock change, no notification.

Why it works across servers. The "queue" is the row lock inside the database, which every server has to go through. No server needs to know about the others. And even if some future code got it wrong, the database rule still refuses negative stock.

How the test proves it. The test starts two separate API instances. It locks the product row itself, sends one order to each instance, and waits until SQL Server confirms both are stuck behind the lock. Then it lets go, so both race at exactly the same moment, on every run:

- Old code: both succeed.
- New code: one success, one refusal, stock 0, one order line.

E. Query performance

Before: 202 queries. The page query loaded every column, including some it didn't need, and then:

```
SELECT TOP(2) [c].[Name] FROM [Categories] AS [c] WHERE [c].[Id] = @id   -- repeated 200 times
```

After: 2 queries. A count, plus this:

```
SELECT p.Id, p.Name, p.Sku, c.Name, p.UnitPrice, p.Stock, p.ReorderThreshold
FROM (SELECT ... FROM Products ORDER BY Id OFFSET @skip ROWS FETCH NEXT @take ROWS ONLY) AS p
INNER JOIN Categories AS c ON p.CategoryId = c.Id
ORDER BY p.Id
```

The database's plan: read products in ID order and stop at 200 rows (no sorting), then look up each category by its key. That's 6 page reads for products and about 3 ms of CPU. A search scans all 2,000 products in about 9 ms; no index can help with "contains" searches.

All SQL above was captured from the API's own log, not written by hand. Full logs are in `release/Release_Evidence/logs/EV-C1…C6`.

F. The refactoring

Job	Before	After
Validate the request	Controller	`OrderService`
Check and take stock	Controller (read, then write)	`OrderService` (one safe statement)
Save the order, transaction	Controller (no transaction)	`OrderService`
Cancel and restock	Controller (racy)	`OrderService` (safe)
Decide on a low-stock alert	Controller	`OrderService`
Send live updates	Controller	`OrderService`, after saving
Turn results into HTTP replies	Mixed in	`OrdersController`: its only job

The safety comes from the database statement and transaction, not from the class structure, so it works no matter how many servers run.

G. Low-stock alert flow

1. An order arrives.
2. Stock is taken off with the safe statement.
3. In the same transaction, we read the new stock and work out what it was before.
4. The order is saved and the transaction commits.
5. A `StockUpdated` message goes to everyone. If stock crossed below the reorder level, a `LowStockAlert` goes too.
6. The browser receives it, and the toast appears on every page.

Delivery: alerts are sent once, after saving. If sending fails, or someone is offline at that moment, that alert is lost (we log it). Guaranteeing delivery would need an "outbox" table; we haven't seen a need for that yet.

H. Verification results

Area	What we ran	Result	Status
Overselling	Last-unit race, two API instances	One success, one refusal	Pass
Stock integrity	40-order burst, rollback tests, database rule, two live API processes, stress script	Never oversold; stock never negative	Pass
Query speed	SQL before and after, timings, plan	202 → 2 queries, 225 → 12 ms	Pass

Area	What we ran	Result	Status
Live updates	Two real browser sessions + 5 SignalR tests	Other browser updates without refresh	Pass (automated browsers; no person-at-keyboard test)
Refactoring	13 API tests, before and after	Identical	Pass
Low-stock alerts	8 rule tests, front-end tests, live browsers	Correct, in every browser	Pass
Backend build	`dotnet build`	0 warnings, 0 errors	Pass
Frontend build	`npm run build`	Succeeds	Pass
All automated tests	`dotnet test`, `ng test`	42/42 and 17/17	Pass
Reconnecting after an outage	Unit tests only	Pass	Partial

On the original code, 9 of the 40 tests failed. Those 9 were the reported bugs.

I. How AI was used

This work was done with **Claude Code** (an AI coding assistant), with the full assessment brief as its prompt. Treat it as AI-assisted work and check it the same way you would check anyone else's.

Where the AI's first idea was wrong, and what caught it:

First idea	What showed it was wrong	Correction
"Overselling means stock goes negative."	The data: stock never went negative; orders simply outnumbered deductions.	Tests now compare units sold against stock taken.
A race-test helper that watched for one specific kind of lock wait	The test timed out: SQL Server reports the second waiter differently.	Count any blocked request.
"`localhost,1433` is the Docker database."	Login failures; the port turned out to belong to SQL Server Express.	Use port 14333 for local tools.
"Hot reload will apply the new migration."	The migration history table hadn't changed.	Restart the API.
Treating a lopsided load-test run as proof	All wins came from one server, because the other was still warming up.	Re-ran warm: wins split 41 / 39.

Why we chose this design:

- **A single safe statement**, rather than "detect conflicts and retry", because nothing ever needs a retry — and a retry is how you get duplicate orders.
- **No caching or indexes** for the list, because the measurements showed the cost was the 202 round trips, not the amount of data.

How it was checked:

- Every bug was reproduced on the original code first.
- Tests run against real SQL Server, not an in-memory fake.
- SQL was captured from the API's own logs.
- Browser behaviour was checked in real Chrome.

J. If traffic doubled, what breaks first?

We measured with 20, then 40, simultaneous users on one test PC, with the API in development mode:

Workload	20 at once	40 at once	What it means
Inventory page	878 req/s, typical 20 ms	1,043 req/s, typical 34 ms	Twice the users, only 19% more throughput: nearing its limit
Orders, all for one product	135 orders/s, typical 140 ms	121 orders/s, typical 326 ms	Twice the users, *no* extra throughput, double the wait

Our answer: popular products break first. Every order for the same product has to wait its turn for that product's lock (about 7 ms each). Double the demand on a best-seller and waiting times double, while throughput stays flat. In a flash sale, requests would start timing out. That's the price of never overselling, and we measured it.

Likely, but not yet proven:

- The API used the most CPU (85%, against 63% for the database). Development-mode logging probably accounts for part of that.
- Live updates multiply with the number of connected boards.
- More than one API server needs a SignalR backplane.

What we'd measure next:

- A production build with quieter logging.
- Orders spread across many products.
- SQL Server wait statistics.
- Hundreds of connected browsers.

K. What's left

1. **More than one API server** → add a SignalR backplane (e.g. Redis); otherwise some users miss updates.
2. **Alerts can be lost** → add an outbox, if guaranteed delivery matters.
3. **Retry after timeout can duplicate an order** → add an idempotency key.
4. **Possible lock-order clash in multi-item orders** → lock products in ID order, or retry.
5. **Hot products queue up** → keep transactions short; load-test a production build.
6. **Editing a product overwrites its stock** (`PUT /api/products`) → require the row version.
7. **Search will slow down as the catalogue grows** → consider full-text search.

8. **Not tested:** a person-at-keyboard two-browser test, a live reconnect after an outage, a staging deployment, a production-build load test, and the lock-clash scenario.
9. **The walkthrough video is not recorded.** The step-by-step guide is in docs/WALKTHROUGH_SCRIPT.md.
10. **Cosmetic:** the low-stock toast overlaps the page-heading counter.

Commits (newest first, nothing rewritten or squashed):

Commit	Change
7ee8eeb	Walkthrough recording guide
6c5bed4	Swagger UI at `/swagger` (development only)
b7e2c5f	Branded release note and test evidence report
4555d20	Root cause analysis and solutions docs
280ac2d	Load generator
1bfb08b	Low-stock toasts
75569d9	Order service refactor
d8329cc	Live updates to everyone
2f076ee	Inventory list speed-up
6157517	Overselling fix
6028cbd	Reliable race test
6bae192	Reproduction tests
fc3c873	Original code, untouched